BIM308 – Web Server Programming – 2nd Assignment

In this assignment, you will extend the previous MVC application by using Entity Framework Core with SQL Server and the Code-First approach. The goal of this assignment is to understand:

- Entity Framework Core basics
- Code-First database creation
- Model relationships
- One-to-Many relationships
- Many-to-Many relationships
- Database operations in ASP.NET Core MVC

1. Creating the Project

Create a new ASP.NET Core Web App (Model-View-Controller) project.

Name the project **BookLibraryEF**.

Run the application to ensure it builds and works correctly.

Install the required NuGet packages:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools

Configure SQL Server connection string inside appsettings.json.

2. Creating the Models

Create a class named **Author**.

Add the following properties:

- AuthorID (int)
- AuthorName (string)
- AuthorInfo (string)
- Navigation Property: A collection of Books written by the Author
- Relationship: **One Author → Many Books**

Create a class named **User**.

Add the following properties:

- UserID (int)
- Name (string)
- Email (string)
- Navigation Property: A collection of Books rented by the user
- Relationship: **Many Users ↔ Many Books**

  A user can rent multiple books.
  A book can be rented by multiple users.

Create a class named **Book**.

Add the following properties:

- BookID (int)
- Title (string)
- ReleaseYear (int)
- Price (double)
- ImageUrl (string)
- Foreign Key: AuthorID
- Navigation Properties:
  - Author
  - A collection of Users who rented the book
- Relationship: **One Book → One Author**, **Many Books → Many Users**

3. Creating the DbContext

Inside the Data folder, create a class named **LibraryContext**. This class must inherit from **DbContext**.

Add DbSet properties:

- DbSet<Book>
- DbSet<Author>
- DbSet<User>

Configure the relationships if needed using **OnModelCreating()**.

4. Database Creation (Code-First)

Use **Code-First migrations** to create the database.

Steps:

1. **Add migration:** Add-Migration InitialCreate
2. **Update Database:** Update-Database

5. Creating the Controller

Create a controller named **LibraryController.**

Add the following actions:

Index Action: Display a list of books from the database. Use **Entity Framework queries** to retrieve the data.

AddBook Action: Create a form to add a new book. Save the data to the database using EF Core**.**

AddUser Action: Create a form to add a new user.

RentBook Action: Allow a user to rent a book.

6. Creating the Views

Create and design “the following pages:

- Index
- AddBook
- AddUser
- RentBook

Use Razor syntax to display the data retrieved from the database.

Each book must display its author information.

Submitting the Assignment

Zip up your project which should be named as **yourlastname_yourID_hw2.zip** and submit it to ESTUOYS platform until March 29 at 23:59.

Only one group member should make submission. Also, please provide a **ReadMe.txt** file that gives information about the group members. Otherwise, your colleague might not receive any points.

Do not forget to put your Project into a container by using the **Dockerfile** that is given to you. Be careful! You will need to modify your Dockerfile and change the last line from "**BookLibrary.dll**" to "**BookLibraryEF.dll**".

Late submission is not allowed by the system. Submissions via e-mail will not be accepted by the instructors and they will be deleted immediately.

7. Routing

Use the default MVC routing configuration.
Run the application.
Access the page using the route.
Verify that the page loads correctly.
Test /Library and observe the result.

8. Execution

When the application runs:

- The page opens without errors.
- The book's information and description is visible.
- At least five books are listed.
- The routing works correctly.

An Example View

Submitting the Assignment

Zip up your project which should be named as **yourlastname_yourID_hw1.zip** and submit it to ESTUOYS platform until March 10 at 23:59.
Only one group member should make submission. Also, please provide a **ReadMe.txt** file that gives information about the group members. Otherwise, your colleague might not receive any points.
Do not forget to put your Project into a container by using the **Dockerfile** that is given to you.
Late submission is not allowed by the system. Submissions via e-mail will not be accepted by the instructors and they will be deleted immediately.